

Probability Model-Based Test Suite Reduction

Pan Liu
Shanghai Business School
liup@sbs.edu.cn

Jun Ai[†]
Beihang University
aijun@buaa.edu.cn

Zhenning(Jimmy) Xu[†]
University of Southern Maine
zhenning.xu@maine.edu

DOI: 10.1145/3127360.3127371

<http://doi.acm.org/10.1145/3127360.3127371>

ABSTRACT

The paper presents a test suite reduction approach based on probability models in regression testing. First, we model the process of software testing by using the joint probability of both test cases and program faults, and propose two probability models for the software's old and new versions, respectively. Then, a theorem combining set operations and probability models is given to implement our approach. Different from the traditional coverage-based test suite reduction methods, the reduced test suite constructed by our approach does not need to cover all test requirements, but also remains the same fault detection capability of the original test suite.

Categories and Subject Descriptors

D.3.3 [Software Engineering]: Testing and Debugging

Keywords

test suite reduction; probability model; regression testing; fault detection capability.

1. INTRODUCTION

The development of large complicated software involves an iterative process of bug fixing, feature enhancement, and regression testing. To detect program faults in the new version of software [1], software authors have designed some new test cases for the modified codes, then incorporated them into the original test suite, resulting in redundant test cases and the growth of testing costs[2]. To decrease regression testing costs, many test suite minimization techniques [3-7] have been proposed by eliminating those redundant test cases which cover test requirements that have been covered by other test cases. Although these techniques can effectively reduce the size of the test suite, experimental results in [8, 9] have shown that some of these techniques severely decreased the Faulty Detection Capability (*FDC*) of the reduced test suite. Thus, some researchers adopted tradeoff methods [10, 11] to reduce the test suite size significantly without losing too much the *FDC*.

Recently, our study [12] has shown that even if test cases cover the same program statements, they may have the different *FDC* of software. Hence, those coverage-based test suite reduction methods may not be the best choice to reduce the size of the test suite. In the process of regression testing, in fact, a new program is constructed by modifying a few codes in the old version of software, which has been tested using the original test suite. Thus, the new test suite consisting of both the original test suite and new test cases cannot expose program faults on those execution paths, where modified statements are not covered.

Based on the relationship between latest and old versions of software, the paper presents a probability model-based method to reduce redundancy of test suite. First, we take test cases as inputs of the program and program faults as its outputs. Then, the process of testing a program can be regarded as a Bayesian network, which can be modeled as the joint probability of both test cases and program faults. For both the original and its modified programs, the joint probability model can be transformed into two conditional probability models to describe the relationship among test cases, executable paths, and program faults on the new and old versions of software, respectively. Then, based on these

probability models, we prove a theorem which can be used to reduce the size of the test suite. Next, an example is given to illustrate the implementation of our approach. The experimental results show that our method can not only effectively reduce the size of the test suite, but also the reduced test subset has the same *FDC* as the original test suite.

The difference between our approach and those coverage-based test suite reduction methods is that the reduced test suite obtained by our approach does not need to cover all test requirements of the modified program. And more importantly, the validity of our approach has been proven by using both set theory and probability models.

The remainder of the paper is constructed as follows: Section 2 gives the motivation of our approach using a simple example. Section 3 introduces a probability model to describe the process of software testing. Section 4 presents a theorem to ensure the validity of the implementation of our approach. Section 5 provides an example for applying our approach in practice. Section 6 studies the related work and section 7 concludes the whole paper.

2. MOTIVATION

There are two control flow graphs shown in Figure 1, where the control flow in Figure 1 (a) is for the original program and that in Figure 1 (b) is for the modified program. Assume that the node 1 is the initial node and the node 12 the final node in two control flow graphs. The modified statements in nodes 3' and 6' in the control flow graph shown in Figure 1 (b). Assume that the test suite is $T = \{t_1, t_2, t_3, t_4, t_5\}$ for the program in the old version, where

- the execution path of t_1 is p_1 : 1-12,
- the execution path of t_2 is p_2 : 1-2-3-5-9-11-12,
- the execution path of t_3 is p_3 : 1-2-3-6-9-11-12,
- the execution path of t_4 is p_4 : 1-2-4-7-10-11-12, and
- the execution path of t_5 is p_5 : 1-2-4-8-10-11-12.

To guarantee the correctness of the amended program in the new version, we design two new test cases t_6 and t_7 whose execution paths are p_6 : 1-2-3-5-9-11-12-4-8-10-11-12 and p_7 : 1-2-3-6-9-11-12-4-7-10-11-12, respectively. Then, the test suite for the program in the new version is $T' = \{t_i | 1 \leq i \leq 7\}$.

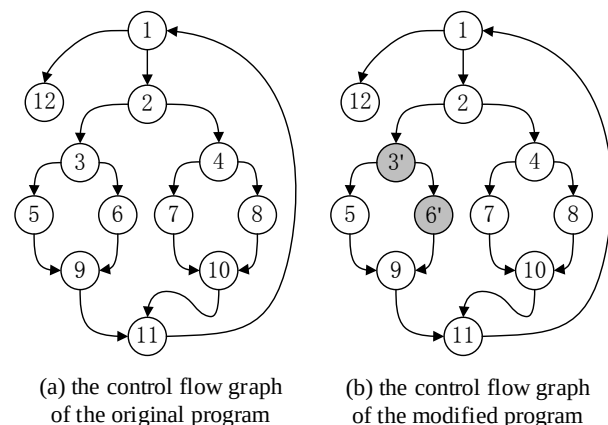


Figure 1. Two control flow graphs.

[†]Corresponding authors: Jun Ai and Zhenning(Jimmy) Xu. Emails: aijun@buaa.edu.cn and zhenning.xu@maine.edu.

From Figure 1, p_1 , p_4 , and p_5 are the same three execution paths in both versions and their corresponding test cases are t_1 , t_4 , and t_5 , respectively. Since test cases in T cannot be used to identify any program faults in the old version of the program, test cases t_1 , t_4 , and t_5 in T' will also fail to detect program faults in execution paths p_1 , p_4 , and p_5 in the new version of the program. Thus, test cases t_1 , t_4 , and t_5 can be removed from T' , and the test subset $\{t_2, t_3, t_6, t_7\}$ retains the same FDC as T' .

Note that in this example, even if there is a program fault that can be identified in the execution path p_8 : 1-2-3-5-9-11-1-2-4-7-10-11-1-12 of the old version of the program, test cases t_1 , t_4 , and t_5 will also fail to detect the fault. Therefore, for the modified version of the program, both the reduced test suite $\{t_2, t_3, t_6, t_7\}$ and $\{t_i | 1 \leq i \leq 7\}$ will have the same FDC.

In the iterative software process, new test cases are still designed for modified code of used in the new versions of the program; hence the unmodified parts of programs don't need to be tested again using test cases designed for the old version when we test the new version. To seek those execution paths through which no program fault can be detected by the test suite designed for the new version of the program, we need to study the relationship among test cases, execution paths, and program faults.

3. PROBABILITY MODEL

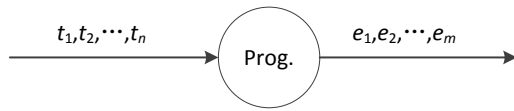


Figure 2. The process of software testing.

Figure 2 shows the software testing process, in which a set of test cases $t_1, t_2, \dots, t_n$ is input into a program; then a set of faults $e_1, e_2, \dots, e_m$ is output from the program. Let T be the set of test cases and E the set of program faults. Then, a Bayesian network can be applied to describe the test process. In particular, a joint probability $P(E, T)$ of both E and T is established. If the test process is successful in identifying the program faults of the program, then $P(E, T) = 1$; otherwise, $P(E, T) = 0$. Therefore, the process of *test suite reduction* is to find a test subset T_1 of T so that the joint probability of both E and T is equal to that of both E and T_1 , which can be formally defined as:

$$\exists T_1 \subset T \bullet P(E, T) = P(E, T_1) \quad (3.1)$$

The formula in (3.1) provides a method to ensure the validity of the test suite reduction process. However, it is hard to directly find a subset T_1 of T which satisfies the formula in (3.1) because there is not a direct relationship between test cases and program faults. A test case must generate a unique execution path, and a program fault may be identified through one or more execution paths, but a test case may not be able to be used to find a program fault. Therefore, we need to take into account the relationship among test cases, program faults, and execution paths. Let H be the set of execution paths. Then, the sample space of software testing is $E \times T \times H$, and there is a relation $R \equiv E \times T \times H$.

Let be $(e_i, t_j, p_l) \in R$, where $e_i \in E$ denotes the i^{th} ($0 \leq i \leq |E|$) fault in E , $t_j \in T$ denotes the j^{th} ($0 \leq j \leq |T|$) test case in T , and $p_l \in H$ denotes the l^{th} ($0 \leq l \leq |H|$) execution path in H . Let $P(e_i, t_j, p_l)$ represent the joint probability of e_i , t_j , and p_l . Its physical interpretation is the probability that t_j can expose e_i on p_l . Therefore, $P(e_i, t_j, p_l) = 0$ indicates that t_j cannot find e_i , or p_l is not an execution path of t_j , or e_i is not found on p_l ; otherwise $P(e_i, t_j, p_l) > 0$. If e_i does not exist, then any test case t_j cannot detect e_i on p_l , that means, $P(e_i, t_j, p_l) = 0$. According to the description of $P(e_i, t_j, p_l)$, we can obtain

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H|} P(e_i, t_j, p_l) = \begin{cases} 0 & E = \emptyset \\ 1 & E \neq \emptyset \end{cases}, \quad (3.2)$$

where $|E|$, $|T|$, and $|H|$ denotes the size of the sets E , T , and H , respectively.

Now let us consider $P(e_i, t_j, p_l)$ in both the old and new versions. Assume that Sp denotes a program in the old version, and Np denotes its new version of the modified program in the new version. Then, the probability $P(e_i, t_j, p_l)$ can be rewritten as two the conditional probabilities $P(e_i, t_j, p_l | Sp)$ and $P(e_i, t_j, p_l | Np)$, where $P(e_i, t_j, p_l | Sp)$ denotes the joint probability of e_i , t_j , and p_l in the Sp , and $P(e_i, t_j, p_l | Np)$ denotes the joint probability of e_i , t_j , and p_l in the Np .

In the iterative software process, the Sp has been tested by its test suite and no fault can be found in the Sp . Hence, for the Sp , there is a formula

$$\sum_{i=1}^{|E_s|} \sum_{j=1}^{|T_s|} \sum_{l=1}^{|H_s|} P(e_i, t_j, p_l | Sp) = 0, \quad (3.3)$$

where E_s , T_s , and H_s denotes the set of program faults, the set of test cases, and the set of execution paths in the Sp , respectively.

For the Np , there is a formula

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H|} P(e_i, t_j, p_l | Np) = \begin{cases} 0 & E = \emptyset \\ 1 & E \neq \emptyset \end{cases}, \quad (3.4)$$

where E , T , and H denote the set of program faults, the set of test cases, and the set of execution paths in the Np , respectively.

Hence, for the Np , if we can find a subset T_1 of T such that the sum of the joint probabilities of program faults, test cases in T_1 , and execution paths is equal to that of program faults, test cases in T , and execution paths, the process of test suite reduction is valid. It can be formally described as

$$\exists T_1 \subset T \bullet \sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H|} P(e_i, t_j, p_l | Np) = \sum_{i=1}^{|E|} \sum_{j=1}^{|T_1|} \sum_{l=1}^{|H|} P(e_i, t_j, p_l | Np) \quad (3.5)$$

Note that for $E = \emptyset$ the formula in (3.5) is correct because any test cases can be deleted from the test suite of the Np . For $E \neq \emptyset$ the formula in (3.5) is also correct because T_1 can also detect all program faults in E . In the rest of the paper, we use two formulas in (3.1) and (3.5) to ensure the validity of test suite reduction techniques.

4. REDUCTION THEOREM

Definition 1 (Mapping Function). A mapping function $f : T \rightarrow H$ denotes that a test case can generate a unique execution path.

Theorem 1. There is $P(e_i, t_j, p_l) = 0$ if there is $f(t_j) \neq p_l$.

Theorem 1 can be proved by both definition 1 and the description of $P(e_i, t_j, p_l)$.

Definition 2 (Change Path and Unchanged Path). Assume that p_1 and p_2 are two execution paths of the test case t in the Sp and the Np , respectively. The path p_2 is called the change path of the Np if both p_1 and p_2 cover different statements. Otherwise, the path p_2 is known as the unchanged path of the Np .

Given that H_1 is the set of changing paths, $H - H_1$ is the set of unchanging paths in the Np ; T_1 is the set of all test cases whose execution paths are the changing paths.

Then, any execution path in H is either a change path or an unchanged path. Hence, there are $H = H_1 \cup (H - H_1)$ and $H_1 \cap (H - H_1) = \emptyset$.

Theorem 2. The test suite T_1 has the same FDC as the test suite T of the Np . Theorem 2 can be formally described as:

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H|} P(e_i, t_j, p_l | Np) = \sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H_1|} P(e_i, t_j, p_l | Np).$$

Proof:

Since there are $H = H_1 \cup (H - H_1)$ and $H_1 \cap (H - H_1) = \emptyset$, we can obtain:

$$\begin{aligned} \sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H|} P(e_i, t_j, p_l | Np) &= \sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H_1|} P(e_i, t_j, p_l | Np) \\ &+ \sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H-H_1|} P(e_i, t_j, p_l | Np) \end{aligned} \quad (4.1)$$

Any $p \in (H - H_1)$ is also a path in Sp according to definition 2, which means that there are $p \in H_s$ and $H - H_1 \subseteq H_s$. Thus, p doesn't contain any modified statements in the Np . For any new test case t in $T - T_s$, there is $f(t) \neq p$ when $p \in H_s$. Hence, according to theorem 1, there is:

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T-T_s|} \sum_{l=1}^{|H-H_1|} P(e_i, t_j, p_l | Np) = 0. \quad (4.2)$$

Since there are $T = (T - T_s) \cup T_s$ and $(T - T_s) \cap T_s = \emptyset$, we obtain:

$$\begin{aligned} \sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H-H_1|} P(e_i, t_j, p_l | Np) &= \sum_{i=1}^{|E|} \sum_{j=1}^{|T-T_s|} \sum_{l=1}^{|H-H_1|} P(e_i, t_j, p_l | Np) \\ &+ \sum_{i=1}^{|E|} \sum_{j=1}^{|T_s|} \sum_{l=1}^{|H-H_1|} P(e_i, t_j, p_l | Np) \end{aligned} \quad (4.3)$$

By (4.2) and (4.3), there is:

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H-H_1|} P(e_i, t_j, p_l | Np) = \sum_{i=1}^{|E|} \sum_{j=1}^{|T_s|} \sum_{l=1}^{|H-H_1|} P(e_i, t_j, p_l | Np) \quad (4.4)$$

According to the formula in (3.3), there must be:

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T_s|} \sum_{l=1}^{|H-H_1|} P(e_i, t_j, p_l | Sp) = 0. \quad (4.5)$$

Hence, for any test case t in T_s , it also cannot find any program fault on any path in $H - H_1$. Thus, we can obtain:

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T_s|} \sum_{l=1}^{|H-H_1|} P(e_i, t_j, p_l | Np) = 0. \quad (4.6)$$

By (4.4) and (4.6), there is:

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H-H_1|} P(e_i, t_j, p_l | Np) = 0. \quad (4.7)$$

By (4.1) and (4.7), we can obtain:

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H|} P(e_i, t_j, p_l | Np) = \sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H_1|} P(e_i, t_j, p_l | Np). \quad (4.8)$$

Since there are $T = (T - T_1) \cup T_1$ and $(T - T_1) \cap T_1 = \emptyset$, the formula in (4.8) can be converted to:

$$\begin{aligned} \sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H|} P(e_i, t_j, p_l | Np) &= \sum_{i=1}^{|E|} \sum_{j=1}^{|T-T_1|} \sum_{l=1}^{|H_1|} P(e_i, t_j, p_l | Np) \\ &+ \sum_{i=1}^{|E|} \sum_{j=1}^{|T_1|} \sum_{l=1}^{|H_1|} P(e_i, t_j, p_l | Np) \end{aligned} \quad (4.9)$$

By theorem 1, there is:

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T-T_1|} \sum_{l=1}^{|H_1|} P(e_i, t_j, p_l | Np) = 0. \quad (4.10)$$

By (4.9) and (4.10), there is:

$$\sum_{i=1}^{|E|} \sum_{j=1}^{|T|} \sum_{l=1}^{|H|} P(e_i, t_j, p_l | Np) = \sum_{i=1}^{|E|} \sum_{j=1}^{|T_1|} \sum_{l=1}^{|H_1|} P(e_i, t_j, p_l | Np). \quad (4.11)$$

Hence theorem 2 is proved. $\square$

By theorem 2, we need to find the test subset T_1 in the test suite reduction.

5. OUR APPROACH

To illustrate our approach, an example is given in Figure 3, in which (a) is an original program, (b) is its modified program, (c) is a test program, and (d) contains a set of test cases. We can observe the output in p_2 or p_3 of the test program in Figure 3 (c) when testing the modified program in Figure 3 (b). The test suite $T = \{t_i \mid 1 \leq i \leq 10\}$ is designed for the modified program in Figure (b). When the modified program in Figure 3 (b) has program faults, we obtain an unexpected output from the test program in Figure 3 (c).

<pre> int getLayers(String st){ int[] layer=new int[st.length()]; int value=0; B₁ for(int i=0; i<st.length(); i++){ char c=st.charAt(i); B₂ if(("".equals(c)) layer[i]=++value; B₃ if(("").equals(c)) layer[i]=--value; B₄ if("".equals(c)) B₅ if(st.charAt(i-1)!='') layer[i]=0; else layer[i]=value+1; } return layer; } </pre>	(a)	<pre> int getLayers(String st){ int[] layer=new int[st.length()]; int value=0; for(int i=0; i<st.length(); i++){ char c=st.charAt(i); if(("".equals(c)) layer[i]=++value; if(("").equals(c)) layer[i]=--value; if("".equals(c)) layer[i]=0; if(st.charAt(i+1)!='') ← p1 layer[i]=0; else layer[i]=value+1; } return layer; } </pre>	(b)
<pre> void test(String st){ try{ system.out.println(getLayer(st)); }catch(Exception ex){ String s=ex.getMessage(); system.out.println(s); ← p3 } } </pre>	(c)	<pre> t₁:(st=""); t₆:(st="*"); t₂:(st=" "); t₇:(st="b.c").d); t₃:(st="a"); t₈:(st="a*).b).c"); t₄:(st="abc"); t₉:(st="a*"); t₅:(st="(a.b)"); t₁₀:(st="a*.b"); </pre>	(d)

Figure 3. An example for illustrating our approach.

Our approach contains the following two steps:

Step 1: Build the test requirement coverage table (shown in Table 1) for the modified program in Figure 3 (b) where TC denotes test cases, B_{iT} and B_{iF} respectively denotes that the judgments are true and false for $1 \leq i \leq 10$, and symbol X denotes that a test case covers B_{iT} or B_{iF} .

Step 2: Select those test cases covering the requirements that represent the modified statement(s) in the modified program. Since the test requirement B_5 contain the statement in p_1 of the modified program shown in Figure 3 (b), according to theorem 2, we need to pick up those test cases from T to cover B_{5T} or B_{5F} in Table 1. Then, the test subset $T_1 = \{t_i \mid 6 \leq i \leq 10\}$ is obtained. Using this verification, we can find the error of string index out of range, which is caused by the statement in p_3 of the test program when testing the modified program by t_9 .

Table 1. The test requirement coverage table of the original program shown in Figure 3 (a).

TC	B_{1T}	B_{1F}	B_{2T}	B_{2F}	B_{3T}	B_{3F}	B_{4T}	B_{4F}	B_{5T}	B_{5F}
t_1		X								
t_2	X	X		X		X		X		
t_3	X	X		X		X		X		
t_4	X	X		X		X		X		
t_5	X	X	X	X	X	X		X		
t_6	X	X		X	X	X	X	X	X	
t_7	X	X		X	X	X	X	X		X
t_8	X	X		X		X	X	X	X	X
t_9	X	X		X		X	X	X		X
t_{10}	X	X		X	X	X	X	X		X

Step 2: Select those test cases covering the requirements that represent the modified statement(s) in the modified program. Since the test requirement B_5 contain the statement in p_1 of the modified program shown in Figure 3 (b), according to theorem 2, we need to pick up those test cases from T to cover B_{5T} or B_{5F} in Table 1. Then, the test subset $T_1 = \{t_i \mid 6 \leq i \leq 10\}$ is obtained. Using this verification, we can find the error of string index out of range, which is caused by the statement in p_3 of the test program when testing the modified program by t_9 .

Different from existing coverage-based test suite reduction methods, the reduced test suite constructed by our approach may not need to cover all requirements in Table 1. For example, the reduced test suite T_1 by our approach does not cover the test requirement B_{2T} .

If we use traditional heuristic methods [3, 8] to construct the reduced test suite of the modified program in Figure 3 (b) the subset $\{t_5, t_8\}$ is obtained. However, both test cases t_5 and t_8 do not expose the error of string index out of range in the modified program in Figure 3 (b). In addition, both the Jeffrey and Gupta method [4] and the RTB method [13] can obtain the test subset $\{t_1, t_5, t_6, t_7, t_8\}$. But, $\{t_1, t_5, t_6, t_7, t_8\}$ also does not detect the error of string index out of range in the modified program in Figure 3 (b). Besides, the Jeffrey and Gupta method retains more redundant test cases in the reduced suite than traditional heuristic methods.

6. EXPERIMENT

To evaluate our approach, we conducted an experiment on the **grep system** [14] with 50 versions to research the efficiency of both the test suite reduction ratio and the fault detection ratio.

6.1 Experimental Object

In this experiment, through the method of mutation testing [15], the grep system was injected with 50 faults into some functions so as to construct 50 versions with faults. Then, we designed 40,450 test cases for all 50 versions and obtained a set of dynamic function call paths as execution paths of test cases for the grep system by viewing the address of called functions in memory when the grep system is running.

The Suite Size Reduction Ratio (SSRR) [4, 13] is defined as:

$$SSRR = \frac{|T| - |T_R|}{|T|} \times 100\%, \quad (6.1)$$

where $|T|$ is the size of the test suite for the new version and $|T_R|$ is the size of the reduced suite.

The Fault Detection Ratio (FDR) [4, 13] is defined as:

$$FDR = \frac{|F|}{|F_R|} \times 100\%, \quad (6.2)$$

where $|F|$ is the number of faults exposed by the original suite and $|F_R|$ is the number of faults detected by the reduced suite.

6.2 Experimental Result

The results of the experiment are shown in Figure 4. From Figure 4 (a), the SSRR is between 0% and 20% on 31 versions, between 20% and 30% on 5 versions, between 40% and 60% on one version, between 90% and 100% on 5 versions, and 100% on 10 versions. To explain the reason for $SSRR=100\%$ on 10 versions, we looked at the called functions in the path and found that those functions with injected faults were not being covered by execution paths of test cases. Hence, according to theorem 2, all those paths can be deleted. Experimental results in Figure 4 (a) indicate that our method can effectively reduce redundant test cases. In Figure 4 (b), both the original test suite and the reduced test suite have the same FDR. In addition, an interesting phenomenon was observed that neither the original test suite nor the reduced test suite can be used to detect injected faults on 10 versions. By comparing Figure 4 (a) with Figure 4 (b), we find that these 10 versions, in which the SSRR of the reduced test suite reaches 100%, are those versions, in which the FDR of the reduced test suite is 0. By calculation, the average test suite reduction ratio of our approach reaches 38.27% for the grep system.

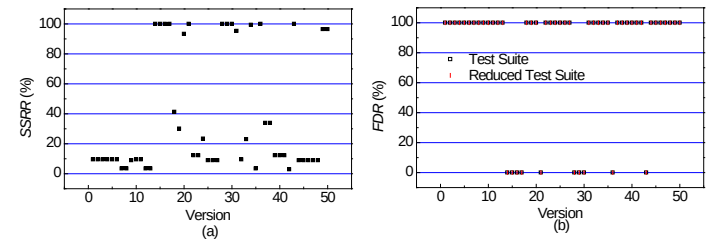


Figure 4. (a) shows the SSRR on 50 versions and (b) shows the FDR both test suite and reduced test suite on 50 versions .

In the following cases, our method will fail:

- 1) When the modified statements are outside all loop or judgment structures, our approach does not work because all executable paths cover those modified statements. In this case, we suggest designing a set of new test cases to test the modified program.
- 2) With an increase in the number of modified statements in the new version of the program, the SSRR of our approach will decline. Thus, if there are lots of statements in a program which are modified, we suggest recognizing the program as a new program to test without the process of test suite reduction

7. RELATED WORK

Work related to the paper can be arbitrarily divided into three categories:

- 1) single coverage criterion-based test suite reduction techniques (that use a coverage criterion to construct a subset of test cases [3,8]),
- 2) multi-coverage criteria-based test suite reduction techniques (that use several coverage criteria to build a subset of test cases [4, 9, 16]), and
- 3) the tradeoff between test cost and the test reduction ratio [10, 11].

Single coverage criterion-based test suite reduction. The early methods focused on the requirement coverage of test cases. Hence, researchers try to seek out a smaller test suite by using some heuristic techniques to satisfy all requirements. For example, heuristic G algorithm was presented by Chvatal [17] to give a priority of selecting

test cases for those uncovered test requirements. Then, Leiserson et al. [18] provided a detailed discussion of this algorithm. Harrold et al. [3] stated HGS algorithm to reduce the test suite. The proposed algorithm first picks up those essential test cases, then the strategy of heuristic G algorithm is adapted to select the other test cases. Chen and Lau [8] presented the GRE algorithm to reduce test suite. This algorithm first removes the 1-to-1 redundant test cases, which cover the same requirements. Then, those test cases that are called essential test cases are selected. Next, a strategy of applying G algorithm is used to pick up test cases. However, in the experiment by Zhong et al. [9], both Heuristic GRE and Heuristic HGS have identical reduction capability of the test suite.

To improve the efficiency of Heuristic GRE, Tallam and Gupta [19] proposed a Delayed-Greedy method. The proposed method uses the concept lattice to reduce test cases. Then, Hutchins et al. [20] and Vokolos et al. [21] carried out some experiments on the Siemens test suite and space programs. Experimental results showed that the Delayed-Greedy method can obtain a smaller test subset than prior heuristics methods.

Multi-coverage criteria-based test suite reduction. To improve the FDC of the reduced test suite, Jeffrey and Gupta [4] suggested adding some redundant test cases selected by other coverage criteria to the reduced test suite. Since the reduced test suite satisfies multiple testing criteria, test suite has a higher FDC than the reduced test suite constructed using one testing criterion. To reduce the number of the test subset constructed by the Jeffrey and Gupta method, Liu et al. [13] presented a test suite reduction approach with Tie-Breaking (RTB) to reduce the test suite. The proposed approach also uses the definition-use pairs criterion to prior select some more important test cases to the reduced test suite.

Harman [22] presented the technique of the multi-objective regression test optimization. Since the process regression testing needs to consider the cost-value trade off, traditional test suite reduction techniques based on requirement coverage are not suitable for this objective. Through several examples, Harman discussed some factors which affect the application of the test suite reduction approaches in the practice. Yoo et al. [23] reduced the test suite by considering these factors that have maximum coverage, minimal execution time, and faulty historical information.

The tradeoff between test cost and the test reduction ratio. Hao et al. [11] presented an approach to address the tradeoff between fault detection loss and the reduction ratio of the test suite. The proposed approach requires testers to first set the maximal fault detection loss for reduced test suite. Then, a proportion of test cases are to be trimmed so that the reduced test suite can satisfy the given suite reduction ratio. Yoo and Harman [24] suggested using multi-objectives to reduce test suite. This approach can also achieve the balance between test cost and the fault detection ratio of the reduced test suite. Gu et al. [25] developed the HATS (Hierarchical Alignment of Two-dimensional Spectra) algorithm to realize the multi-objective test suite reduction. The approach defines a weighted factor to address the tradeoff between the requirement coverage and the reduced test suite size.

8. Conclusion

In the paper, we have presented a novel approach to reduce redundant test cases based on probability models in regression testing. To begin with, we simulated the process of software testing using a Bayesian network technique, then constructed the joint probability of both the test suite and the faulty set. Next, a probability model of test suite reduction in regression testing was given. Then, we built two formulas by using the joint probability for the original program and the modified program. To ensure the efficiency of our approach, we presented a theorem to select those test cases which have the same FDC as the original test suite. Then, an example is given to illustrate the implementation of our approach in practice. Finally, we also conducted an experiment on the

open source grep system to evaluate the efficiency of our approach. A major difference between traditional coverage-based test suite reduction methods and our approach is that our approach does not need the reduced test suite to satisfy all test requirements. The main contribution of this paper is to introduce the probability model into regression testing and to put forward the approach of test suite reduction based on probability models.

9. ACKNOWLEDGMENTS

The authors thank Ph.D. Ruizhi Gao of the University of Texas at Dallas. He gave some valuable suggestions for this paper. This work is supported by National Natural Science Foundation of China (NSFC) under the grant (No. 61502299), Science and technology key project of Jiangxi Province (No. 20142BBE50015).

10. REFERENCES

- [1] ROTHERMEL, G., UNTCH, R.H., CHU, C., and HARROLD, M.J., 2001. Prioritizing test cases for regression testing. *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING* 27, 10, 929-948. DOI= <http://doi.org/10.1109/32.962562>.
- [2] YOO, S. and HARMAN, M., 2012. Regression testing minimization, selection and prioritization: a survey. *Software Testing, Verification and Reliability* 22, 2, 67-120. DOI= <http://onlinelibrary.wiley.com/enhanced/exportCitation/doi/10.1002/stvr.430>.
- [3] HARROLD, M.J., GUPTA, R., and SOFFA, M.L., 1993. A methodology for controlling the size of a test suite. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 2, 3, 270-285. DOI= <https://doi.org/10.1145/152388.152391>.
- [4] JEFFREY, D. and GUPTA, N., 2007. Improving fault detection capability by selectively retaining test cases during test suite reduction. *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING* 33, 2. DOI= <https://doi.org/10.1109/TSE.2007.18>.
- [5] HSU, H.-Y. and ORSO, A., 2009. MINTS: A general framework and tool for supporting test-suite minimization. In *2009 IEEE 31st International Conference on Software Engineering* IEEE, 419-429. DOI= <https://doi.org/10.1109/ICSE.2009.5070541>.
- [6] BLUE, D., SEGALL, I., TZOREF-BRILL, R., and ZLOTNICK, A., 2013. Interaction-based test-suite minimization. In *Proceedings of the 2013 International Conference on Software Engineering* IEEE Press, 182-191. DOI= <https://doi.org/10.1109/ICSE.2013.6606564>.
- [7] EMAM, S.S. and MILLER, J., 2015. Test case prioritization using extended digraphs. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 25, 1, 6. DOI= <https://doi.org/10.1145/2789209>.
- [8] CHEN, T.Y. and LAU, M.F., 1998. A simulation study on some heuristics for test suite reduction. *Information and Software Technology* 40, 13, 777-787. DOI= [https://dx.doi.org/10.1016/s0950-5849\(98\)00094-9](https://dx.doi.org/10.1016/s0950-5849(98)00094-9).
- [9] ZHONG, H., ZHANG, L., and MEI, H., 2008. An experimental study of four typical test suite reduction techniques. *Information and Software Technology* 50, 6, 534-546. DOI= <https://doi.org/10.1016/j.infsof.2007.06.003>.
- [10] MCMASTER, S. and MEMON, A., 2008. Call-stack coverage for gui test suite reduction. *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING* 34, 1, 99-115. DOI= <https://doi.org/10.1109/ISSRE.2006.19>.
- [11] HAO, D., ZHANG, L., WU, X., MEI, H., and ROTHERMEL, G., 2012. On-demand test suite reduction. In *Proceedings of the 34th International Conference on Software Engineering* IEEE Press, 738-748. DOI= <https://doi.org/10.1109/ICSE.2012.6227144>.

- [12] LIU, P., 2014. An efficient reduction approach to test suite. In *Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD), 2014 15th IEEE/ACIS International Conference on* IEEE, 1-5. DOI= <https://doi.org/10.1109/SNPD.2014.6888743>
- [13] LIN, J.-W. and HUANG, C. -Y., 2009. Analysis of test suite reduction with enhanced tie-breaking techniques. *Information and Software Technology* 51, 4, 679-690. DOI= <https://doi.org/10.1016/j.infsof.2008.11.004>
- [14] NAVARRO, G., 2001. NR - grep: a fast and flexible pattern - matching tool. *Software: Practice and Experience* 31, 13, 1265-1312. DOI= <http://onlinelibrary.wiley.com/doi/10.1002/spe.411/abstract>
- [15] NATELLA, R., COTRONEO, D., DURAES, J.A., and MADEIRA, H.S., 2013. On fault representativeness of software fault injection. *IEEE TRANSACTIONS ON SOFTWARE ENGINEERING* 39, 1, 80-96. DOI= <https://doi.org/10.1109/TSE.2011.124>
- [16] HARMAN, M., 2011. Making the Case for MORTO: Multi Objective Regression Test Optimization. In *ICST Workshops Citeseer*, 111-114. DOI= <https://doi.org/10.1109/ICSTW.2011.60>
- [17] HSU, H.-Y. and ORSO, A., 2009. MINTS: A general framework and tool for supporting test-suite minimization. In *2009 IEEE 31st International Conference on Software Engineering* IEEE, 419-429. DOI= <https://doi.org/10.1109/ICSE.2009.5070541>
- [18] CORMEN, T.H., 2009. *Introduction to algorithms*. MIT press.
- [19] TALLAM, S. and GUPTA, N., 2006. A concept analysis inspired greedy algorithm for test suite minimization. *ACM SIGSOFT Software Engineering Notes* 31, 1, 35-42. DOI= <https://doi.org/10.1145/1108792.1108802>
- [20] HUTCHINS, M., FOSTER, H., GORADIA, T., and OSTRAND, T., 1994. Experiments of the effectiveness of dataflow and controlflow-based test adequacy criteria. In *Proceedings of the 16th international conference on Software engineering* IEEE Computer Society Press, 191-200. DOI= <https://doi.org/10.1109/ICSE.1994.296778>
- [21] VOKOLOS, F.I. and FRANKL, P.G., 1998. Empirical evaluation of the textual differencing regression testing technique. In *Software Maintenance, 1998. Proceedings., International Conference On* IEEE, 44-53. DOI= <https://doi.org/10.1109/ICSM.1998.738488>
- [22] HARMAN, M., 2011. Making the Case for MORTO: Multi Objective Regression Test Optimization. In *ICST Workshops Citeseer*, 111-114. DOI= <https://doi.org/10.1109/ICSTW.2011.60>
- [23] YOO, S., NILSSON, R., and HARMAN, M., 2011. Faster fault finding at Google using multi objective regression test optimisation. In *8th European Software Engineering Conference and the ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE'11), Szeged, Hungary*
- [24] YOO, S. and HARMAN, M., 2010. Using hybrid algorithm for pareto efficient multi-objective test suite minimisation. *Journal of Systems and Software* 83, 4, 689-701. DOI= <https://doi.org/10.1016/j.jss.2009.11.706>
- [25] GU, Q., TANG, B., and CHEN, D.-X., 2011. A test suite reduction technique for partial coverage of test requirements. *Jisuanji Xuebao(Chinese Journal of Computers)* 34 5, 879-888.